Universidad de La Habana
Facultad de Matemática y Computación

Ingeniería de Software

Seminario de Arquitecturas de Software:

Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Equipo: Base de Batos

Integrante	Pregunta	Contribución individual
Alex Leonardo Cuervo Grillo	1	Análisis de comunicación, connascencia y tolerancia a fallos.
Héctor Duarte Vázquez	2	Análisis de elasticidad, escalabilidad y consistencia en SBA.
Alejandro López Castro	3	Análisis del Workflow Event Pattern y resiliencia.
Fabio Hernández Piloto	4	Análisis de granularidad y descomposición mediante bounded contexts.
Gabriel Pérez Suárez	5	Comparación de consistencia, despliegue y trade-offs.
Bryan Ernesto Moya Aquino	6	Análisis de SOA, reutilización y orquestación. Confección del documento

Fecha de entrega: 13 de noviembre de 2026

Repositorio público del proyecto:
`https://github.com/el-M0ya/Base_de_Batos`

Índice

1. Problema 1: Análisis Comparativo de Comunicación y Acoplamiento 1
1.1. Escenario de Dominio . . . 1
1.2. Impacto en el Acoplamiento Dinámico y Connascencia (*Connascence*) . . . 1
1.3. Tolerancia a Fallos y Consistencia de Datos (ACID vs. BASE) . . . 3
1.3.1. Análisis Detallado por Arquitectura: . . . 3

2. Problema 2: Diseño para Elasticidad Extrema y Consistencia de Datos 6
2.1. Por qué SBA frente a microservicios . . . 6
2.2. Mitigación de colisiones en cachés replicadas . . . 7
2.3. Trade-offs . . . 8

3. Problema 3: Análisis de Fallos y Resiliencia en Arquitecturas Dirigidas por Eventos 9
3.1. Impacto del Fallo Repetitivo en la Topología Broker . . . 9
3.2. Solución Arquitectónica: *Workflow Event Pattern* . . . 10
3.2.1. Componentes Adicionales del Patrón . . . 10
3.2.2. Flujo de Mitigación . . . 10
3.3. *Trade-offs* e Implicaciones de la Solución . . . 11

4. Problema 4: El Dilema de la Granularidad y el Acoplamiento en Microservicios 13
4.1. ¿Cuál fue el error fundamental? . . . 13
4.2. Nueva estrategia de descomposición . . . 14
4.3. Ejemplo: la operación de contratación . . . 14
4.4. Trade-offs de la nueva descomposición . . . 16
4.5. Conclusión . . . 17

5. Problema 5: Comparación de Atributos de Calidad y Trade-offs 17
5.1. Planteamiento y respuesta breve . . . 17
5.2. Qué significa consistencia absoluta en finanzas . . . 18
5.3. Comparación de los estilos . . . 18
5.4. Análisis por requisito . . . 18
5.4.1. Consistencia de datos . . . 18
5.4.2. Despliegue rápido e independiente . . . 19
5.4.3. Experiencia mixta del equipo . . . 19
5.5. Trade-off y recomendación . . . 19
5.6. Diagramas conceptuales . . . 19
5.7. Conclusión . . . 19

6. Problema 6: El Legado de SOA y su Influencia en Arquitecturas Modernas 20
6.1. Reutilización en SOA y en Microservices Architecture . . . 21
6.2. Reutilización de la lógica de negocio y reutilización operativa . . . 21
6.3. Cuándo sigue siendo conveniente la orquestación . . . 22
6.3.1. Ejemplo: procesamiento de un pedido . . . 22
6.4. Conclusión . . . 23

Declaración de Uso de la Inteligencia Artificial 24

1. Problema 1: Análisis Comparativo de Comunicación y Acoplamiento

Enunciado. Un sistema de subastas en línea requiere que, al finalizar una subasta, se realicen múltiples acciones: notificar al ganador, procesar el pago y actualizar el inventario. En un diseño con Microservicios, un arquitecto podría optar por una coreografía de eventos (comunicación asíncrona) o una orquestación mediante un servicio mediador. En un diseño con Service-Based Architecture, estas acciones probablemente se ejecutarían dentro de un mismo servicio de dominio de grano grueso. Basado en los conceptos de *connascence* (connascencia) y los *trade-offs* de la comunicación síncrona vs. asíncrona, analice:

a) ¿Cómo afecta cada enfoque (orquestación vs. coreografía en microservicios, y servicios de grano grueso en SBA) al acoplamiento dinámico entre los componentes del sistema?

b) ¿Qué implicaciones tiene cada diseño para la tolerancia a fallos y la consistencia de los datos (ACID vs. BASE) si el servicio de pago falla justo después de que se notifica al ganador?

Respuesta.

1.1. Escenario de Dominio

En un sistema de subastas en línea, la finalización de una subasta requiere la coordinación de tres acciones fundamentales:

1. **Notificar al ganador** (envío de credenciales/confirmación).
2. **Procesar el pago** (cargo automático a la tarjeta de crédito registrada).
3. **Actualizar el inventario** (marcar el artículo como vendido/asignado).

A continuación, se presenta el análisis comparativo entre tres enfoques arquitectónicos: **Microservicios con Coreografía (Broker)**, **Microservicios con Orquestación (Mediador)** y **Service-Based Architecture (SBA)**.

1.2. Impacto en el Acoplamiento Dinámico y Connascencia (*Connascence*)

El acoplamiento dinámico representa la dependencia que existe entre los componentes durante la ejecución del sistema, especialmente en términos de comunicación, disponibilidad y coordinación temporal. Por su parte, la *connascencia* expresa el grado de interdependencia entre dos componentes, es decir, cuánto deben conocerse o cambiar conjuntamente para mantener la corrección del sistema. Ambos conceptos permiten evaluar el nivel de dependencia que introduce cada enfoque arquitectónico (Richards and Ford, 2020).

1. **Microservicios con Coreografía (Topología Broker / Asíncrono):**
   - **Acoplamiento Dinámico: Bajo acoplamiento directo.** Los servicios productores y consumidores se comunican indirectamente mediante un *Message Broker*. El servicio de subastas publica un evento, como `AuctionEndedEvent`, sin conocer directamente a sus consumidores ni esperar sus respuestas. Esto permite que cada servicio procese los eventos de manera independiente y que no necesiten estar disponibles simultáneamente. Sin embargo, existe una dependencia respecto al broker y a los contratos de los eventos intercambiados.
   - **Análisis de Connascencia:**

- *Connascencia Estática de Nombre, Tipo y Significado:* El productor y los consumidores deben coincidir en los nombres de los eventos y sus atributos, en los tipos de datos y en el significado de cada valor. Por ejemplo, todos deben interpretar correctamente el propósito y la estructura de `AuctionEndedEvent`. Un cambio incompatible en el contrato del evento puede exigir modificaciones en los consumidores.
- *Connascencia Dinámica de Ejecución y Tiempo:* Aunque la comunicación asíncrona reduce la dependencia temporal inmediata, no elimina toda dependencia relacionada con el orden o el momento de procesamiento. Por ejemplo, el pago debe completarse antes de confirmar definitivamente la venta del artículo. Por tanto, pueden existir restricciones de ejecución y temporales dentro del flujo de negocio.

2. **Microservicios con Orquestación (Topología Mediador / Coordinada):**

   - **Acoplamiento Dinámico: Moderado a Alto.** Un servicio coordinador (*Auction Mediator*) conoce a los participantes y dirige explícitamente el flujo de ejecución, determinando cuándo solicitar una notificación, procesar el pago y actualizar el inventario. Esto introduce una dependencia directa respecto al mediador y a los contratos de los servicios participantes.
   - **Análisis de Connascencia:**
     - *Connascencia de Algoritmo:* La secuencia de operaciones y las reglas que determinan el flujo del proceso quedan centralizadas en el orquestador. Si cambia la lógica de coordinación, puede ser necesario modificar este componente.
     - *Connascencia de Ejecución y Tiempo:* La coordinación exige respetar determinadas relaciones de orden entre las operaciones. Si el mediador utiliza llamadas síncronas (HTTP/gRPC), también depende de la disponibilidad inmediata de los servicios invocados y acumula sus latencias. Si emplea comunicación asíncrona, puede reducir esta dependencia temporal inmediata, aunque debe gestionar la correlación de mensajes y el estado del proceso.

3. **Service-Based Architecture (Arquitectura Basada en Servicios, SvcBA):**

   - **Acoplamiento Dinámico: Bajo a nivel de red entre los componentes internos; mayor dependencia estructural dentro del servicio.** Al agrupar las funcionalidades de subastas, pagos e inventario dentro de un servicio de dominio de grano grueso (ej. *Auction & Order Service*), sus interacciones pueden realizarse mediante llamadas a funciones en memoria (*in-process*), sin necesidad de comunicación de red entre ellas.
   - **Análisis de Connascencia:**
     - *Connascencia Estática de Nombre, Tipo y Estructura:* Los módulos internos pueden compartir modelos, tipos y estructuras de datos, lo que introduce dependencias de compilación y dificulta su evolución completamente independiente.
     - *Dependencia de Datos Compartidos:* Cuando varios servicios de dominio utilizan una base de datos compartida, pueden depender de sus esquemas y estructuras. Esto introduce acoplamiento respecto a los datos, aunque las operaciones internas de un mismo servicio puedan ejecutarse sin comunicaciones distribuidas.

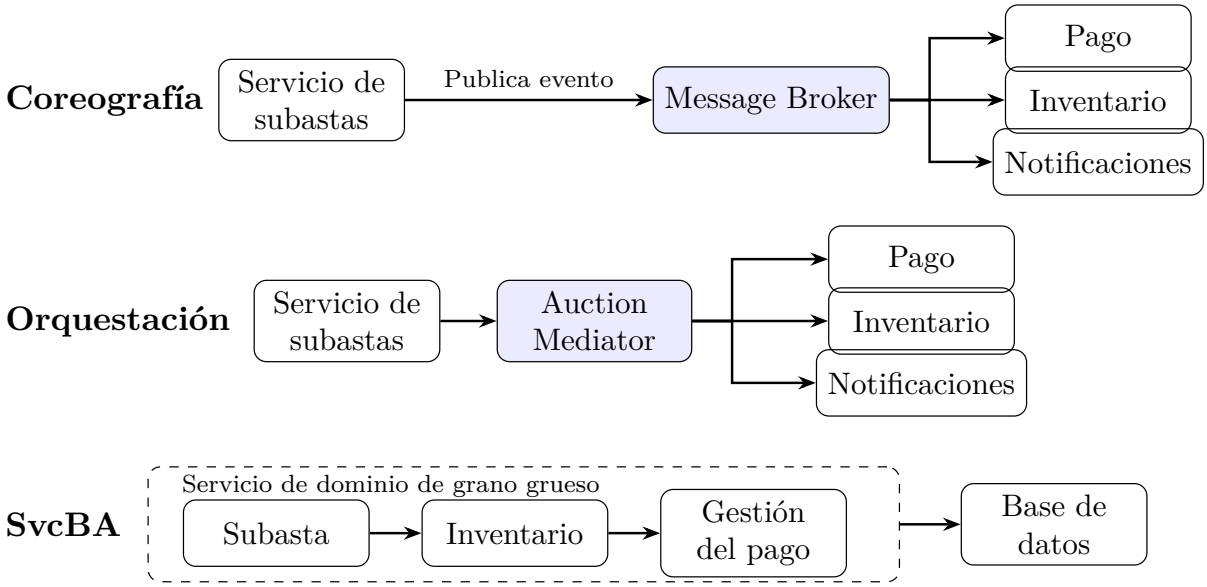


Figura 1: Comparación de los mecanismos de comunicación y los límites de los componentes en los tres enfoques arquitectónicos.

1.3. Tolerancia a Fallos y Consistencia de Datos (ACID vs. BASE)

Escenario de Fallo: *El servicio de pago falla justo después de que se ha notificado al ganador.*

1.3.1. Análisis Detallado por Arquitectura:

1. **Microservicios con Coreografía (Transacciones Locales y Consistencia Eventual):**

   - **Comportamiento:** Cada microservicio administra sus propios datos y ejecuta transacciones ACID dentro de su base de datos local. Sin embargo, cuando una operación de negocio involucra varios servicios, no existe una transacción ACID global que garantice la confirmación o reversión conjunta de todos ellos. Por ello, se suele recurrir a la consistencia eventual y al patrón **Saga por Coreografía** (Ford et al., 2021).
   - **Resolución del Fallo:** Cuando el servicio de pago no puede completar el cobro, publica un evento `PaymentFailedEvent`. Los servicios interesados reaccionan ejecutando las compensaciones correspondientes. Por ejemplo, el servicio de inventario puede liberar la reserva del artículo y el servicio de subastas actualizar el estado de la operación para indicar que no se completó.

     Si la notificación al ganador ya fue enviada, no es posible revertir ese efecto externo. El servicio de notificaciones deberá enviar un nuevo mensaje informando que el pago no pudo procesarse y que la operación fue cancelada. Para evitar esta situación, es preferible que la notificación definitiva se emita únicamente después de confirmar el pago.

     Cada consumidor debe ser capaz de procesar un mismo evento de forma idempotente, ya que la mensajería puede provocar entregas repetidas. También deben utilizarse mecanismos que aseguren la publicación fiable de los eventos después de confirmar las transacciones locales, como el patrón *Transactional Outbox* (Ford et al., 2021).

2. **Microservicios con Orquestación (Transacciones Locales y Coordinación de Saga):**

Criterio	**Coreografía (Broker)**	**Orquestación (Mediador)**	**Service-Based (SvcBA)**
Consistencia	Transacciones ACID locales en cada servicio. La operación global suele alcanzar consistencia eventual mediante eventos y compensaciones (Saga).	Transacciones ACID locales y consistencia eventual entre servicios. El mediador coordina el proceso, pero no garantiza atomicidad global.	Puede garantizar ACID para las operaciones cubiertas por una misma transacción de base de datos. Los efectos externos quedan fuera de esa transacción.
Gestión del fallo de pago	Se publica un evento de fallo (`PaymentFailedEvent`) y los servicios ejecutan las acciones compensatorias necesarias.	El mediador detecta el fallo y dirige explícitamente las operaciones de compensación.	Puede ejecutarse un `ROLLBACK` de las modificaciones locales no confirmadas. Los efectos externos requieren compensaciones específicas.
Notificación previa	Si ya fue enviada, no puede deshacerse. Se requiere una notificación correctiva. Es preferible esperar la confirmación del pago.	El mediador puede controlar el momento de envío. Si la notificación ya se produjo, debe coordinar su corrección.	Debe diferirse hasta después del *commit*. Un `ROLLBACK` no puede retirar un correo o deshacer un cargo externo.
Puntos de fallo	El broker puede convertirse en una dependencia crítica. Requiere alta disponibilidad, replicación y mecanismos de recuperación.	El mediador puede convertirse en cuello de botella o punto único de fallo si no dispone de replicación y recuperación de estado.	La base de datos compartida o el servicio de dominio pueden ser puntos críticos y requieren mecanismos de alta disponibilidad.

Tabla 1: Comparación de consistencia y tolerancia a fallos ante el fallo del servicio de pago.

- **Comportamiento:** Al igual que en la coreografía, cada microservicio ejecuta sus propias transacciones ACID. Las operaciones que involucran varios servicios requieren coordinación y pueden presentar consistencia eventual. La diferencia es que un componente central, el *Orquestador*, mantiene el control explícito de la secuencia y del estado del proceso.
- **Resolución del Fallo:** Si el servicio de pago devuelve una respuesta de error, el orquestador detiene la secuencia normal y ejecuta las acciones compensatorias correspondientes. Por ejemplo, puede solicitar al servicio de inventario liberar la reserva del artículo y al servicio de subastas actualizar el estado de la operación. Una vez completadas estas acciones, solicita al servicio de notificaciones informar al usuario sobre el fallo.

  El orquestador facilita el seguimiento del estado global de la operación y permite definir explícitamente las acciones que deben realizarse ante cada fallo. Sin embargo, estas compensaciones no equivalen a un `ROLLBACK` global: son operaciones de negocio independientes que intentan contrarrestar los efectos de transacciones previamente confirmadas.

  El mediador debe conservar el estado del proceso para poder recuperarse ante una interrupción. Además, debe implementarse con mecanismos de disponibilidad y tolerancia a fallos, evitando que su caída detenga indefinidamente todas las operaciones coordinadas (Ford et al., 2021).

3. **Service-Based Architecture (Transaccionalidad ACID Local):**

   - **Comportamiento:** En una arquitectura basada en servicios, los servicios de dominio de grano grueso pueden compartir una base de datos o utilizar esquemas parcialmente compartidos. Cuando las actualizaciones de la subasta, el inventario y el registro de la orden se realizan dentro de una misma transacción sobre una base de datos relacional, es posible garantizar las propiedades ACID para esas modificaciones.
   - **Resolución del Fallo:**
     - Si el fallo ocurre antes de confirmar la transacción local, la base de datos puede ejecutar un `ROLLBACK` de las modificaciones no confirmadas, evitando que el inventario y el registro de la orden queden en un estado inconsistente.
     - Sin embargo, si el procesamiento del pago implica una pasarela externa de tarjetas de crédito, esta operación no forma parte de la transacción local de la base de datos. Por tanto, un `ROLLBACK` no puede deshacer un cargo que ya haya sido realizado. Para estos casos deben utilizarse mecanismos como la autorización y posterior captura del pago o, si el cobro ya se efectuó, una operación compensatoria de anulación o reembolso.
     - **Regla Crítica de Diseño:** Las notificaciones externas (email/SMS) deben enviarse únicamente después de que se haya confirmado el estado correspondiente de la operación mediante *commit*. Para garantizar que la publicación de estos mensajes no se pierda después de confirmar la transacción, puede utilizarse el patrón *Transactional Outbox*.

La siguiente figura resume el flujo de gestión del pago y las acciones compensatorias que deben realizarse cuando la operación no puede completarse:

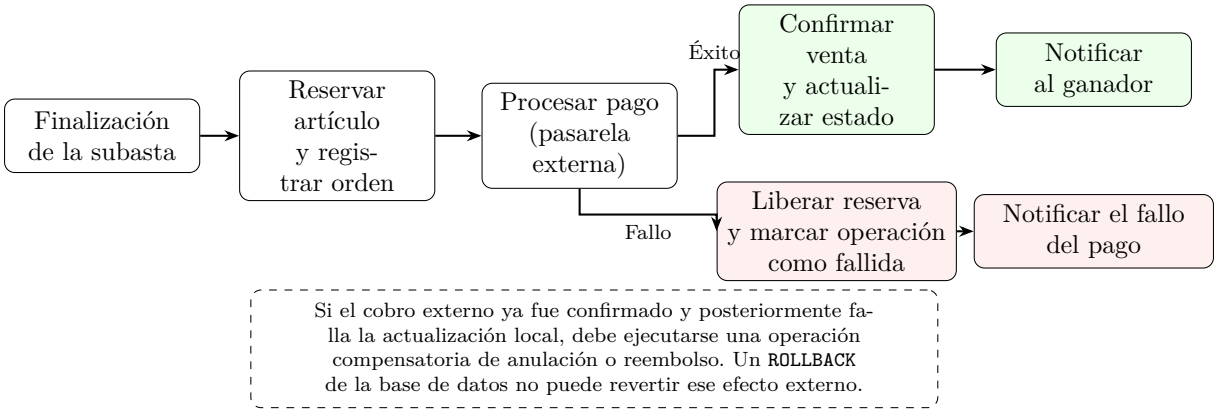


Figura 2: Flujo de confirmación del pago y acciones compensatorias ante un fallo.

2. Problema 2: Diseño para Elasticidad Extrema y Consistencia de Datos

Enunciado. Considere el sistema de venta de entradas para conciertos descrito en el libro (Capítulo 15). Durante el lanzamiento de entradas para un artista popular, el sistema experimenta un aumento repentino de usuarios de cientos a decenas de miles en segundos. Se le pide como arquitecto diseñar la solución.

Justifique por qué una *Space-Based Architecture* sería una elección más adecuada que una *Microservices Architecture* para este escenario específico, enfocándose en las características de elasticidad y escalabilidad.

Sin embargo, la *Space-Based Architecture* introduce el problema de las colisiones de datos en cachés replicadas. Describa cómo un arquitecto podría mitigar el riesgo de colisiones de datos en el contexto de la venta de asientos numerados, donde la consistencia es crucial. ¿Qué mecanismos o *trade-offs* consideraría?

Respuesta.

El sistema de venta de entradas para conciertos es un caso de carga altamente variable: el tráfico puede pasar de cientos a decenas de miles de usuarios en segundos. Para ese escenario, una *Space-Based Architecture* (SBA) es más adecuada que una arquitectura de microservicios porque ataca directamente el cuello de botella que limita la elasticidad: la base de datos central y las llamadas síncronas entre servicios Richards and Ford (2020).

2.1. Por qué SBA frente a microservicios

- **Escalabilidad y elasticidad.** La SBA organiza el sistema en *processing units* que se agregan o retiran dinámicamente, sin depender de una base de datos central en la ruta crítica. Los datos de trabajo se mantienen en *data grids* en memoria, replicados entre unidades, y un *virtualized middleware* coordina el acceso y el balanceo Richards and Ford (2020). Los *data pumps* sincronizan de forma asíncrona con la base de datos persistente, de modo que esta deja de ser el cuello de botella. Esto permite absorber picos repentinos y reducir la capacidad cuando pasa la demanda.

- **Microservicios.** Este estilo optimiza el despliegue independiente y la delimitación por *bounded contexts*, pero cada servicio conserva su propia base de datos y la comunicación suele ser síncrona. Ante un pico de 100×, hay que escalar muchos servicios y sus bases de datos, y la contención se traslada a la red y a la persistencia Richards and Ford (2020). La elasticidad es posible, pero más lenta y costosa de operar.

- **Adecuación al problema.** Para la venta de entradas, la prioridad es la elasticidad ex-

trema y la concurrencia sobre la independencia de despliegue. Por eso la SBA es la opción principal; los microservicios pueden reservarse para contextos de back-office (pagos, catálogo, notificaciones) Richards and Ford (2020).

Tabla 2: Comparación para el pico de venta de entradas.

Criterio	**Space-Based Architecture**	**Microservicios**
Elasticidad	Unidades de procesamiento dinámicas; escala casi lineal	Escala servicio por servicio y también sus bases de datos
Escalabilidad	Sin base de datos central en la ruta crítica; data grid en memoria	Posible, pero con contención de red y persistencia
Consistencia	Eventual por defecto; requiere mecanismos para datos críticos	Mayor control local, pero transacciones distribuidas complejas
Despliegue	Menos unidades, más gruesas	Despliegue independiente por servicio
Complejidad	Alta: coordinación, replicación y sincronización	Alta: observabilidad, contratos y datos distribuidos

La Figura 3 representa la organización de los componentes de una Space-Based Architecture y el mecanismo de sincronización entre los datos en memoria y el almacenamiento persistente.

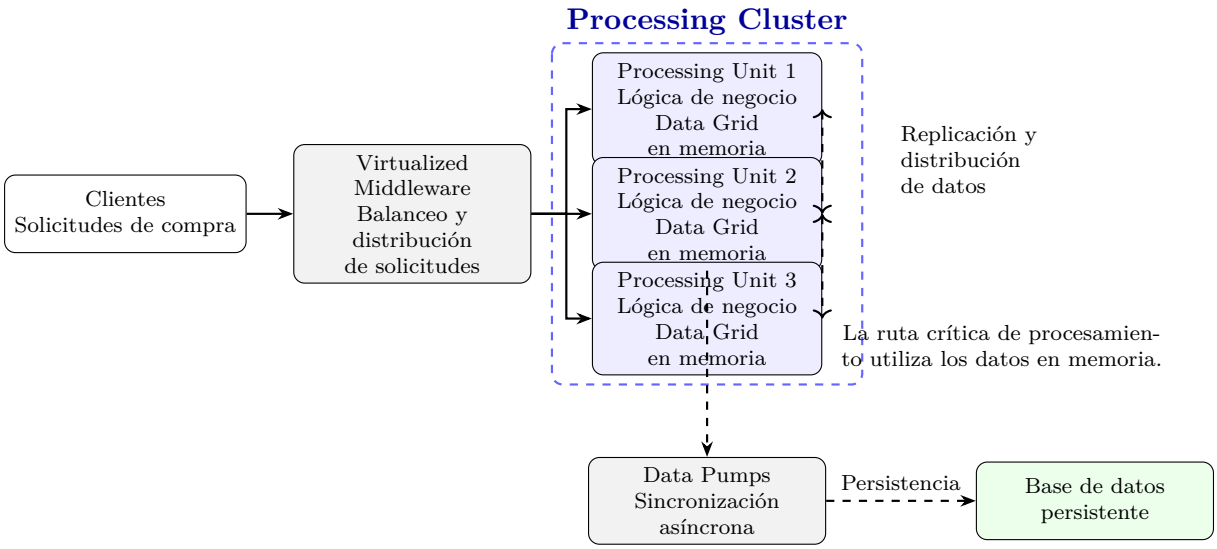


Figura 3: Estructura conceptual de una Space-Based Architecture.

2.2. Mitigación de colisiones en cachés replicadas

El riesgo aparece cuando dos unidades de procesamiento actualizan el mismo asiento en réplicas distintas: la última escritura puede pisar a la otra y provocar sobreventa. Para evitarlo, el arquitecto debe tratar la reserva de asientos como un dato crítico, no como un dato eventualmente consistente Ford et al. (2021).

- **Particionar por evento/asiento y asignar un único dueño.** Los asientos de un evento se dividen en particiones (por ejemplo, por fila o por rango de asientos) y cada partición tiene una unidad líder que procesa las escrituras. Así se evita que dos réplicas escriban el mismo asiento Ford et al. (2021).

- **Transición atómica con control de versión.** El asiento se modela como una máquina de estados DISPONIBLE → RESERVADO → VENDIDO. La reserva se hace con *compare-and-set* o bloqueo optimista sobre la versión; si la versión cambió, la operación se rechaza o se reintenta. Para alta contención se puede usar un bloqueo pesimista o una cola por asiento que serialice las solicitudes.
- **Consistencia fuerte solo en la ruta crítica.** La reserva final puede apoyarse en replicación síncrona por quórum, consenso (Raft/Paxos) o un *ledger* autoritativo de asientos. El caché replicado se usa para lecturas y catálogo, no como fuente de verdad del inventario Ford et al. (2021).
- **Reservas con expiración e idempotencia.** Se separa la reserva temporal (*hold*) de la confirmación de pago. La reserva tiene TTL y las solicitudes llevan clave de idempotencia para evitar duplicados. Si el pago falla, se libera el asiento.
- **Saga y consistencia eventual para el resto.** Pago, notificación y actualización de inventario secundario se coordinan con una saga con compensaciones; CQRS/Event Sourcing pueden aportar auditoría y reconstrucción del estado Ford et al. (2021).

2.3. Trade-offs

- **Consistencia vs. latencia/disponibilidad.** Los mecanismos de consenso, quórum y bloqueos reducen la elasticidad y aumentan la latencia de la reserva; son aceptables solo para el inventario de asientos, no para todo el sistema.
- **Consistencia eventual vs. fuerte.** Notificaciones, catálogo y recomendaciones pueden ser eventuales; la venta de un asiento numerado no.
- **Complejidad operativa.** La SBA y la coordinación distribuida añaden réplicas, reconciliación, monitoreo y recuperación ante fallos. Una alternativa híbrida es usar SBA para la capa elástica de venta y un servicio de reservas fuertemente consistente para el inventario.
- **Evitar 2PC en todo el flujo.** La consistencia fuerte se limita al asiento y se usan sagas para el resto; así se evita un acoplamiento global que reduciría la disponibilidad Ford et al. (2021).

La decisión de diseño es, entonces, una SBA para la elasticidad, con una ruta crítica de reserva de asientos fuertemente consistente y particionada; el resto del sistema puede permanecer eventualmente consistente.

La Figura 4 ilustra cómo se controla la concurrencia cuando dos usuarios intentan reservar simultáneamente un mismo asiento numerado:

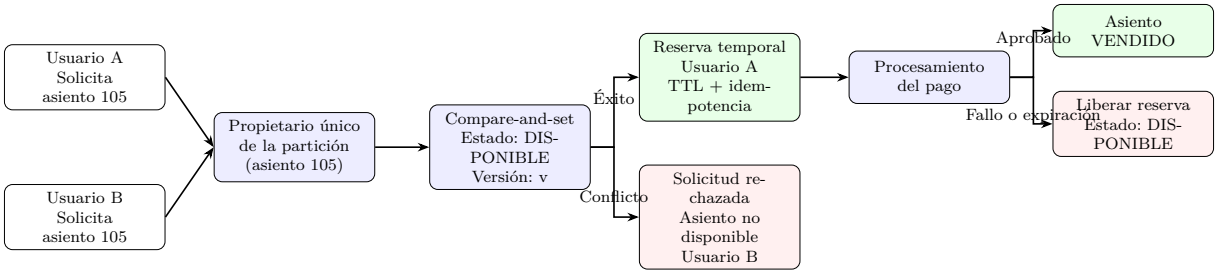


Figura 4: Control de concurrencia y ciclo de vida de una reserva de asiento numerado.

3. Problema 3: Análisis de Fallos y Resiliencia en Arquitecturas Dirigidas por Eventos

Enunciado. En una *Event-Driven Architecture* con topología de Broker, un procesador de eventos (*Event Processor*) falla repetidamente al procesar un tipo específico de mensaje debido a un error en los datos.

Describa el impacto de este fallo en el flujo general del sistema y en la experiencia del usuario final, considerando que no hay un mediador central.

Proponga una solución arquitectónica utilizando el *Workflow Event Pattern* (Patrón de Eventos de Flujo de Trabajo) descrito en el Capítulo 14. Explique detalladamente cómo este patrón mitiga el problema y qué componentes adicionales (como el *workflow processor* y el *dashboard*) serían necesarios, así como los *trade-offs* que introduce (ej. procesamiento fuera de orden).
Respuesta.

3.1. Impacto del Fallo Repetitivo en la Topología Broker

En una *Event-Driven Architecture* (EDA) que utiliza la **topología de Broker**, la ausencia de un componente central de orquestación o mediación implica que el flujo de ejecución depende de la publicación y suscripción de eventos entre los distintos *Event Processors*. Cada procesador reacciona a los mensajes que recibe y produce nuevos eventos como resultado de su procesamiento (Richards and Ford, 2020).

Cuando un procesador falla repetidamente al intentar procesar un mensaje debido a datos corruptos o malformados (*poison pill message*), el impacto puede manifestarse en varios niveles del sistema:

1. **Bloqueo de Cola y Consumo de Recursos (*Poison Pill Effect*):**

   Si el sistema utiliza políticas de reintentos que vuelven a entregar indefinidamente el mismo mensaje defectuoso, el *Event Processor* puede quedar atrapado intentando procesarlo sin éxito. Esto consume recursos de procesamiento y reduce la capacidad de atender otros mensajes. El problema es especialmente crítico cuando se requiere mantener el orden de procesamiento dentro de una cola o partición (FIFO), ya que los mensajes posteriores pueden quedar bloqueados mientras no se resuelva el mensaje defectuoso.

2. **Interrupción del Flujo de Eventos e Inconsistencia de los Procesos:**

   En la topología Broker, los eventos de un procesador pueden desencadenar nuevas operaciones en otros procesadores. Si uno de ellos falla antes de completar su procesamiento y publicar el evento correspondiente, los componentes posteriores no reciben la información que necesitan para continuar.

   Como consecuencia, una operación de negocio puede quedar parcialmente completada. Por ejemplo, una solicitud podría haber sido registrada, pero no llegar a actualizar el inventario o generar la notificación correspondiente. Si no existen mecanismos adicionales de detección, recuperación o compensación, el sistema puede permanecer en ese estado hasta que el problema sea resuelto.

3. **Impacto en la Experiencia del Usuario Final (UX):**

   Debido al carácter asíncrono de EDA, el cliente puede recibir una confirmación de recepción de la solicitud (por ejemplo, una respuesta HTTP `202 Accepted`) sin que esto signifique que la operación haya finalizado correctamente.

   Si el error no se gestiona adecuadamente, el usuario puede experimentar:

   - Un estado de “Procesando” indefinido en la interfaz visual.

- Falta de retroalimentación sobre el fallo real de la operación.
- Información desactualizada en las vistas de datos (*read models*) al no recibirse los eventos necesarios para actualizarlas.

3.2. Solución Arquitectónica: *Workflow Event Pattern*

Para resolver el problema sin perder la capacidad de respuesta y el desacoplamiento de la topología Broker, se propone implementar el ***Workflow Event Pattern***. Este patrón permite delegar el tratamiento de los errores a un componente especializado, evitando que el procesador original interrumpa su actividad mientras intenta resolver un mensaje defectuoso. Su funcionamiento se basa en tres mecanismos fundamentales: delegación, reparación y, cuando sea necesario, intervención humana (Richards and Ford, 2020, pp. 197–200).

3.2.1. Componentes Adicionales del Patrón

- **Canal de Errores (*Error Event Channel*):**

  Canal de mensajería adicional al que el *Event Processor* envía los mensajes que no pudo procesar correctamente. Junto con el mensaje original, se transmite información sobre la excepción producida para facilitar su diagnóstico, así como los datos necesarios para identificar y recuperar el mensaje original.

  Puede implementarse mediante una cola de mensajes persistente. Una *Dead Letter Queue* (DLQ) también puede utilizarse como mecanismo de aislamiento de mensajes fallidos, pero no sustituye la lógica de reparación del patrón.

- ***Workflow Processor*:**

  Es el componente encargado de recibir los errores delegados e identificar sus causas. Analiza el mensaje y determina si el problema puede corregirse automáticamente mediante reglas programáticas.

  Cuando encuentra una solución, modifica los datos defectuosos y vuelve a publicar el mensaje corregido en la cola original. De esta forma, el *Event Processor* lo recibe nuevamente como un mensaje nuevo e intenta procesarlo otra vez.

  Si no consigue determinar cómo reparar el error, deriva el mensaje a otro canal destinado a la intervención manual.

- ***Workflow Dashboard* / Consola de Intervención:**

  Es una interfaz administrativa que recibe los mensajes que no pudieron corregirse automáticamente. Permite a una persona autorizada inspeccionar el mensaje y la información del error, corregir manualmente los datos y volver a enviar el mensaje a su cola original para que sea procesado nuevamente.

  Este mecanismo permite resolver errores que no pueden solucionarse mediante reglas automáticas, sin necesidad de intervenir directamente sobre el procesador original.

3.2.2. Flujo de Mitigación

El funcionamiento del patrón puede describirse mediante la siguiente secuencia:

1. El *Event Processor* recibe un mensaje e intenta procesarlo. Durante la ejecución se produce una excepción debido a un error en los datos.

2. En lugar de permanecer intentando procesar el mismo mensaje, el procesador delega el error mediante un mensaje asíncrono al *Workflow Processor*, utilizando el canal de errores.

Posteriormente, continúa con el procesamiento del siguiente mensaje disponible en la cola original.

3. El *Workflow Processor* recibe el mensaje defectuoso, analiza la excepción e intenta determinar qué modificación debe realizarse para corregir los datos.

4. Si consigue repararlo automáticamente, modifica el mensaje y lo vuelve a enviar a la cola original. El procesador de eventos lo recibe nuevamente e intenta procesarlo, esta vez con los datos corregidos.

5. Si no puede identificar una corrección automática, el *Workflow Processor* envía el mensaje a un segundo canal destinado a la intervención humana. El *Workflow Dashboard* permite a un operador inspeccionarlo y corregirlo manualmente.

6. Una vez corregido, el operador vuelve a enviar el mensaje a la cola original, generalmente utilizando un mecanismo de respuesta asociado al mensaje (*reply-to*), para que continúe el procesamiento normal.

De esta manera, el procesador original no queda bloqueado intentando resolver mensajes defectuosos, mientras que los errores pueden recuperarse mediante reparaciones automáticas o intervención humana. El patrón mejora la resiliencia y mantiene la capacidad de respuesta del sistema sin introducir un mediador central que coordine todas las operaciones (Richards and Ford, 2020, pp. 197–199).

La Figura 5 representa los componentes y el flujo de recuperación descritos anteriormente:

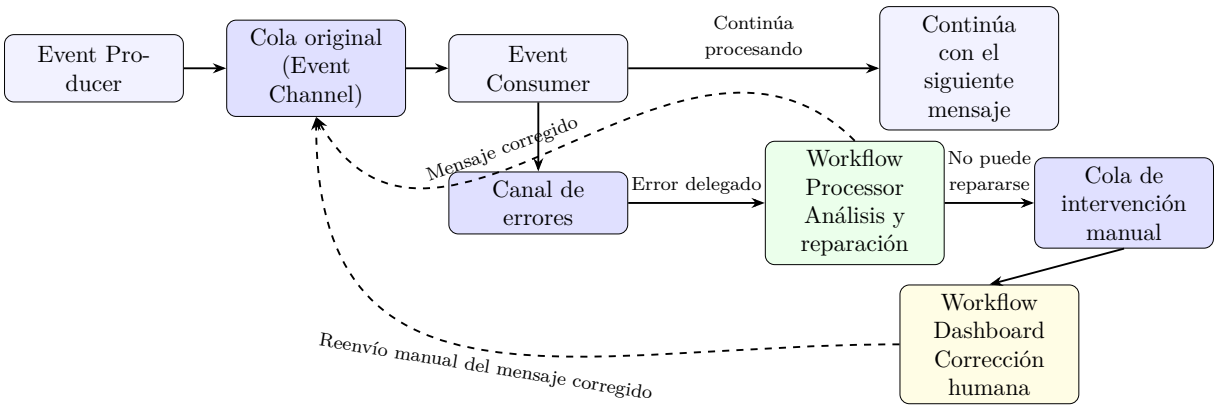


Figura 5: Funcionamiento del Workflow Event Pattern y sus mecanismos de recuperación.

3.3. *Trade-offs* e Implicaciones de la Solución

La adopción del *Workflow Event Pattern* mejora la resiliencia y la capacidad de respuesta del sistema ante errores de procesamiento. Sin embargo, introduce algunas compensaciones arquitectónicas que deben considerarse:

Procesamiento Fuera de Orden (*Out-of-Order Processing*): Cuando un mensaje defectuoso es delegado al *Workflow Processor*, los mensajes posteriores pueden continuar procesándose. Si el mensaje original se corrige y se vuelve a publicar en la cola, será recibido como un mensaje nuevo, por lo que puede procesarse después de otros mensajes que originalmente debían sucederlo.

Esto puede ser problemático cuando existe una dependencia de orden entre los eventos. Por ejemplo, si se procesan operaciones financieras de una misma cuenta, una venta de acciones puede tener que realizarse antes de una compra posterior para mantener la corrección del estado de la cuenta.

Para mitigar este problema, puede conservarse temporalmente una cola FIFO de los mensajes posteriores que compartan el mismo contexto de ordenación (por ejemplo, el identificador de cuenta). Una vez que el mensaje defectuoso haya sido corregido y procesado correctamente, se liberan los mensajes retenidos respetando su orden original. De esta manera, se preserva el orden dentro del contexto afectado sin detener necesariamente el procesamiento de los demás mensajes del sistema (Richards and Ford, 2020, pp. 199–200).

Incremento de Complejidad Operativa: La solución requiere infraestructura y componentes adicionales, como canales de errores, colas de intervención manual, almacenamiento de mensajes y el *Workflow Processor*. También es necesario desarrollar el *Dashboard* y establecer mecanismos de seguimiento y recuperación de los mensajes.

Además, los consumidores deben considerar que un mensaje puede procesarse más de una vez, por lo que resulta conveniente diseñar operaciones idempotentes y mantener la trazabilidad de los mensajes durante su delegación y reprocesamiento.

Latencia en la Consistencia Eventual: La reparación automática y, especialmente, la intervención humana introducen demoras adicionales antes de que el mensaje pueda procesarse correctamente. Durante ese intervalo, los componentes que dependen del evento pueden mantener temporalmente información desactualizada o estados incompletos.

Por tanto, aunque el patrón permite recuperar los mensajes y continuar el flujo de ejecución, no garantiza que todos los componentes alcancen inmediatamente un estado consistente. El tiempo necesario para converger dependerá de la complejidad del error y del mecanismo utilizado para resolverlo.

La Figura 6 ilustra una estrategia para preservar el orden de procesamiento dentro de un mismo contexto, sin bloquear los eventos pertenecientes a otros contextos:

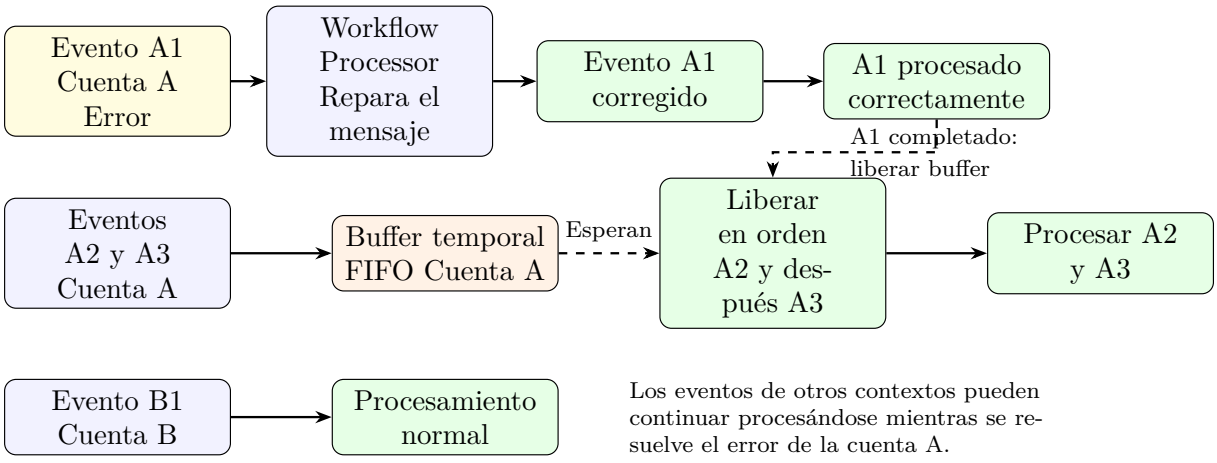


Figura 6: Preservación del orden de procesamiento mediante una cola temporal FIFO por contexto.

4. Problema 4: El Dilema de la Granularidad y el Acoplamiento en Microservicios

Enunciado. Un equipo de desarrollo está migrando un sistema monolítico de gestión de recursos humanos a Microservicios. Inicialmente, crean servicios muy pequeños para cada entidad (Empleado, Departamento, Nómina, Beneficios). Pronto descubren que completar una operación simple, como contratar a un nuevo empleado, requiere una compleja coreografía de llamadas síncronas entre casi todos los servicios, lo que resulta en un alto acoplamiento y una fragilidad similar a la del monolito original.

Basado en los principios de *bounded context* y las guías de granularidad del Capítulo 17, ¿cuál fue el error fundamental en el diseño de los servicios?

Proponga una nueva estrategia de descomposición. ¿Cómo aplicaría los conceptos de propósito, transacciones y coreografía para redefinir los límites de los servicios y reducir el acoplamiento? Proporcione un ejemplo de cómo agruparía los servicios para la operación de contratación.
Respuesta.

4.1. ¿Cuál fue el error fundamental?

Un *bounded context* (contexto delimitado) es un límite dentro del cual un modelo de dominio y su vocabulario tienen un significado coherente. Dentro de ese límite se definen las reglas y responsabilidades propias de una parte del negocio. En una arquitectura de microservicios, estos contextos ayudan a identificar límites candidatos para los servicios y a establecer quién es responsable de cada dato y comportamiento. No significa que un contexto no pueda interactuar con el resto del sistema, sino que debe hacerlo mediante contratos explícitos y sin compartir indiscriminadamente su modelo interno (Richards and Ford, 2020).

El error fundamental fue crear un servicio por cada entidad (Empleado, Departamento, Nómina y Beneficios), en vez de identificar primero las capacidades del negocio y sus límites naturales. Este enfoque, conocido como la *trampa de la entidad*, convierte cada tabla o entidad del modelo de datos en un servicio independiente. El resultado son componentes demasiado pequeños, con poca autonomía funcional, que necesitan comunicarse constantemente para completar operaciones del negocio.

La contratación de un empleado es un proceso de negocio que, con el diseño inicial, quedó distribuido entre varios servicios. Esto introduce dos problemas principales:

1. **Cadenas de llamadas síncronas y acoplamiento dinámico:** cada servicio espera la respuesta del siguiente. Si uno falla o se demora, la operación completa puede fallar o quedar bloqueada. Esta dependencia introduce connascencia dinámica de ejecución y tiempo: los servicios deben estar disponibles y responder en un orden determinado. Es importante precisar que una cadena de llamadas síncronas no constituye, en sentido estricto, una coreografía de eventos; es una coordinación distribuida mediante comunicación request-response. En la práctica, los servicios terminan comportándose como un único sistema fuertemente acoplado, pero con latencias y puntos de fallo adicionales debido a la red.

2. **Límites transaccionales mal definidos:** el diseño hace que una operación de contratación intente modificar datos que pertenecen a varios servicios y bases de datos. Esto dificulta mantener las invariantes del negocio, ya que cada servicio confirma sus cambios de forma independiente. Sin embargo, no significa que el empleado, el departamento, la nómina y los beneficios deban modificarse siempre en una única transacción. La decisión correcta consiste en identificar qué datos deben cambiar atómicamente para preservar una misma invariante y mantenerlos dentro del mismo límite transaccional. Las operaciones que pertenecen a otros contextos pueden completarse posteriormente mediante eventos.

Por tanto, el problema no es que los servicios sean pequeños por sí mismos, sino que sus límites no representan responsabilidades cohesivas del negocio. La necesidad frecuente de coordinar transacciones y llamadas entre ellos es una señal de que la descomposición debe revisarse.

4.2. Nueva estrategia de descomposición

La solución consiste en dividir el sistema según las capacidades del negocio y no según las entidades o tablas de la base de datos. Para establecer los límites se aplican las tres guías de granularidad descritas por Richards y Ford (2020): propósito, transacciones y coreografía (Richards and Ford, 2020).

1. **Propósito:** cada servicio debe encargarse de una capacidad de negocio coherente. "Empleado" o "Departamento" son conceptos del dominio, pero no necesariamente representan por sí solos una responsabilidad que justifique un servicio. En cambio, gestionar el ciclo de vida del personal, administrar la estructura organizativa o gestionar la compensación son propósitos más completos.

2. **Transacciones:** los datos que deban modificarse juntos para preservar una invariante del negocio deben permanecer dentro del mismo límite transaccional. Por ejemplo, si una contratación exige registrar al empleado, crear su contrato y establecer su asignación inicial a un departamento, estas modificaciones pueden realizarse en una transacción local de Gestión de Personal. Esto no obliga a incluir también el alta en nómina y beneficios en esa misma transacción: esas operaciones pueden pertenecer a otro contexto y realizarse después de confirmar la contratación.

   La asignación a un departamento debe validarse conforme a las reglas del negocio. Sin embargo, el catálogo de departamentos puede seguir teniendo un propietario independiente; Gestión de Personal puede conservar una referencia al identificador del departamento y una proyección local para las consultas que no requieran consistencia inmediata.

3. **Coreografía:** cuando una capacidad debe reaccionar a un hecho ocurrido en otro contexto, la comunicación mediante eventos asíncronos reduce la dependencia temporal y evita cadenas innecesarias de llamadas síncronas. Por ejemplo, después de completar una contratación, Gestión de Personal publica un evento que puede ser consumido por Nómina y Beneficios. Cada servicio procesa el evento según sus propias reglas. La coreografía debe utilizarse donde aporte desacoplamiento; no significa que toda comunicación entre servicios tenga que ser asíncrona.

Cada servicio mantiene la propiedad de sus datos y su propia base de datos. Cuando necesita consultar información de otro contexto sin requerir una respuesta inmediata, puede mantener una proyección local actualizada mediante eventos. Esto reduce las consultas remotas repetitivas, aunque introduce la posibilidad de que esa copia esté temporalmente desactualizada.

4.3. Ejemplo: la operación de contratación

Para este escenario se proponen tres servicios. La entidad Departamento requiere distinguir dos responsabilidades: administrar el catálogo y la estructura organizativa, y asignar un empleado a un departamento durante la contratación. La primera pertenece a Estructura Organizacional; la segunda forma parte del proceso de Gestión de Personal. No se trata de duplicar la propiedad del departamento, sino de mantener un propietario del dato y, cuando sea necesario, una proyección local para consulta.

1. **Gestión de Personal (Empleado, contrato y asignación):** se encarga de contratar empleados, administrar sus contratos, actualizar su estado laboral y mantener su asignación a un departamento.

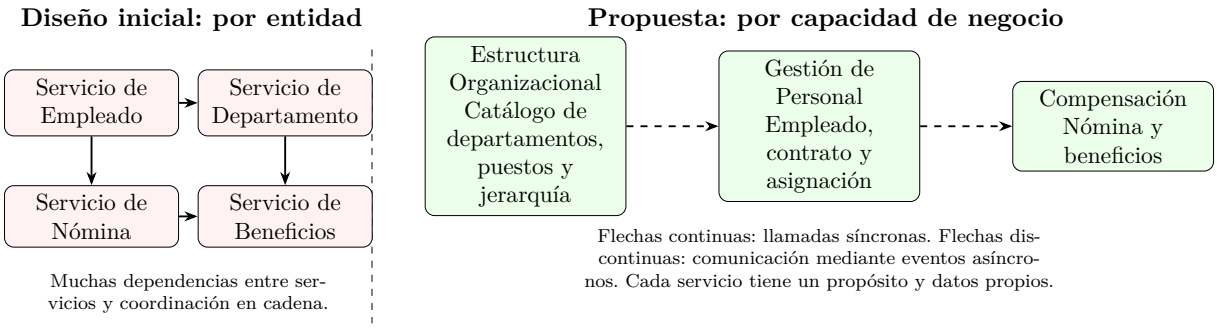


Figura 7: Comparación entre una descomposición basada en entidades y otra basada en capacidades de negocio. Fuente: elaboración propia a partir de Richards y Ford (2020).

2. **Compensación (Nómina y Beneficios):** administra el perfil salarial, el cálculo de nóminas, las deducciones y los beneficios laborales. En esta propuesta se agrupan ambas capacidades porque se asume que forman parte de un mismo ámbito de gestión de la compensación. Si en la organización tienen reglas, responsables y ciclos de vida suficientemente independientes, podrían separarse en dos contextos distintos.

3. **Estructura Organizacional (catálogo de Departamentos):** administra los departamentos, los puestos y la jerarquía de la empresa. Es el propietario de esta información y publica eventos cuando se producen cambios relevantes.

La contratación funcionaría de la siguiente manera:

1. Gestión de Personal recibe la solicitud y valida la información necesaria, incluida la asignación al departamento. Para consultas habituales puede utilizar una proyección local del catálogo de departamentos, actualizada mediante eventos publicados por Estructura Organizacional. Si una regla exige conocer el estado más reciente del departamento, debe definirse un mecanismo explícito de validación que tenga en cuenta la posible demora de esa proyección.

2. En una única transacción local, Gestión de Personal registra al empleado, crea su contrato, establece su asignación al departamento y guarda el evento `EmpleadoContratado` en una tabla de mensajes pendientes. Esta tabla forma parte de la misma base de datos y transacción. Así, el registro de la contratación y la intención de publicar el evento se confirman juntos. Un proceso independiente, denominado *Outbox Publisher*, lee los mensajes pendientes y los publica en el broker, reintentando el envío ante fallos. Este mecanismo evita perder el evento si el servicio cae después del *commit* y antes de publicarlo; sin embargo, puede producir entregas duplicadas, por lo que los consumidores deben ser idempotentes (Ford et al., 2021).

3. Compensación consume el evento y crea el perfil correspondiente en Nómina e inicia la inscripción en los Beneficios. Si el servicio está temporalmente indisponible, el mensaje permanece pendiente en el mecanismo de mensajería y podrá procesarse cuando se recupere, de acuerdo con la política de persistencia y reintentos configurada.

4. Como el evento puede entregarse más de una vez, Compensación debe procesarlo de forma idempotente. Puede registrar los identificadores de eventos ya procesados y aplicar restricciones de unicidad, de modo que una misma contratación no genere perfiles duplicados en nómina o beneficios.

5. Si Compensación no puede completar el alta por una regla del negocio (por ejemplo, porque el salario propuesto está fuera del rango permitido para el puesto), publica un evento

`AltaEnCompensacionRechazada`. Gestión de Personal recibe el evento y actualiza el estado correspondiente, por ejemplo, a *Pendiente de revisión de compensación*. La contratación no se elimina automáticamente: se requiere que Recursos Humanos revise y resuelva la situación. Esta decisión supone que la creación de la relación laboral puede confirmarse antes de completar la configuración de compensación; si el alta salarial fuera una condición obligatoria para que la contratación sea válida, esa regla tendría que verificarse antes de confirmar la contratación.

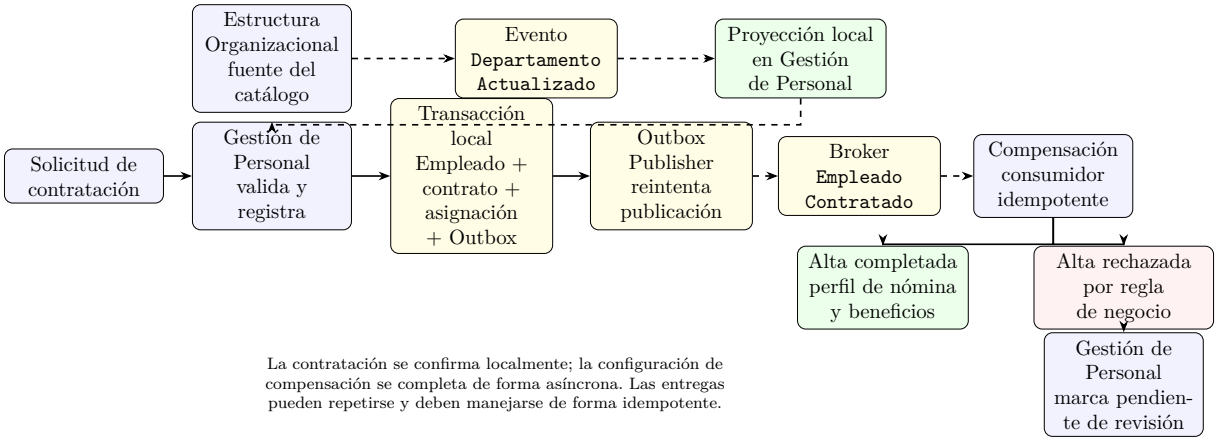


Figura 8: Flujo de contratación con transacción local, publicación mediante Outbox y procesamiento asíncrono. Fuente: elaboración propia.

Esta solución sustituye la cadena de llamadas síncronas por una transacción local seguida de un evento. Gestión de Personal no necesita esperar a que Compensación esté disponible para confirmar el registro de la contratación. No obstante, la indisponibilidad de Compensación puede retrasar el alta salarial y la inscripción en beneficios; el desacoplamiento evita bloquear el proceso inicial, pero no elimina la dependencia de negocio.

4.4. Trade-offs de la nueva descomposición

La propuesta reduce la coordinación síncrona y establece límites más coherentes, pero introduce algunos compromisos:

Consistencia eventual: Gestión de Personal puede registrar al empleado antes de que Compensación cree su perfil. Durante ese intervalo, la contratación existe, pero el alta en nómina y beneficios está pendiente. Esto puede ser aceptable si las reglas del negocio permiten completar esas tareas antes del primer pago. En caso contrario, se necesita una validación previa o un estado explícito que impida considerar el proceso completamente finalizado.

Complejidad operativa: El patrón Outbox requiere almacenar y publicar los mensajes pendientes, gestionar reintentos y supervisar los mensajes que no puedan enviarse. También deben controlarse las duplicaciones y los errores de procesamiento en los consumidores.

Duplicación de datos de consulta: La proyección local del catálogo de departamentos evita consultas remotas constantes, pero puede estar temporalmente desactualizada. Es necesario definir cómo se actualiza y qué reglas pueden tomar decisiones con esa información.

Autonomía frente a coordinación: Los servicios pueden desplegarse y recuperarse de manera independiente, pero un fallo de Compensación puede retrasar los procesos posteriores y acumular mensajes pendientes. Se necesitan mecanismos de monitoreo, reintentos y alertas.

Si una operación de negocio exigiera coordinar transacciones locales de varios servicios y además necesitara acciones compensatorias ante fallos, podría utilizarse el patrón *Saga*. Sin embargo, el evento de rechazo descrito anteriormente no constituye por sí solo una Saga: es un mecanismo de notificación y recuperación. Solo habría una Saga si se definiera un proceso distribuido con pasos confirmados y operaciones compensatorias para contrarrestar sus efectos cuando fuera necesario (Ford et al., 2021).

4.5. Conclusión

El error fundamental fue confundir la granularidad de los microservicios con la separación de las entidades del modelo de datos. La solución consiste en definir los límites arquitectónicos a partir de las capacidades del negocio, los contextos delimitados y las operaciones que necesitan compartir transacciones.

Al agrupar la contratación, el contrato y la asignación del empleado dentro de Gestión de Personal, mantener el catálogo de departamentos en Estructura Organizacional y tratar Nómina y Beneficios como una capacidad de Compensación, se reduce la cantidad de comunicaciones necesarias y se conserva la autonomía de los procesos que pertenecen a contextos diferentes.

La coreografía mediante eventos permite conectar estos servicios sin introducir cadenas extensas de llamadas síncronas. En definitiva, una buena arquitectura de microservicios no se consigue creando la mayor cantidad posible de servicios pequeños, sino encontrando límites que representen correctamente las responsabilidades y los flujos de trabajo del negocio.

5. Problema 5: Comparación de Atributos de Calidad y Trade-offs

Enunciado. Un director de tecnología (CTO) le pide que elija un estilo arquitectónico para un nuevo sistema de procesamiento de transacciones financieras. Los requisitos clave son: 1) Consistencia de datos absoluta (ACID), 2) Capacidad de desplegar nuevas características de negocio de forma independiente y rápida, y 3) Un equipo de desarrollo con experiencia mixta.

Compare *Service-Based Architecture* y *Microservices Architecture* en función de estos tres requisitos. ¿Cuál de las dos arquitecturas es más adecuada para cumplir con el requisito de consistencia de datos y por qué?

¿Cómo se ve afectada la capacidad de despliegue independiente (requisito 2) en cada estilo? ¿Qué *trade-offs* debe aceptar el CTO si prioriza la consistencia sobre la agilidad de despliegue?
Respuesta.

5.1. Planteamiento y respuesta breve

Respuesta. Se comparan Service-Based Architecture (SBA) y Microservices Architecture (MSA) ante tres requisitos: consistencia transaccional ACID, despliegue independiente y rápido, y un equipo con experiencia mixta. Para un sistema cuyo requisito dominante es que cada operación financiera preserve invariantes de forma inmediata, SBA suele ser la opción inicial más adecuada: concentra las operaciones que deben confirmarse juntas en un servicio de dominio de grano grueso y permite que estas compartan una base relacional y una transacción local. Esta conclusión depende de diseñar límites coherentes; una base compartida por sí sola no garantiza integridad, aislamiento ni buen diseño. MSA favorece más la autonomía y la evolución independiente, pero sus bases de datos separadas hacen que una transacción que cruza servicios deje de ser una transacción ACID local. Se puede coordinar con sagas, mensajes y acciones compensatorias, aunque eso normalmente ofrece consistencia eventual y más complejidad operativa, no atomicidad instantánea global Fowler (2015); Amazon Web Services (n.d.); Microsoft (n.d.).

5.2. Qué significa consistencia absoluta en finanzas

ACID describe propiedades de transacciones: atomicidad (todo o nada), consistencia (se preservan reglas del dominio), aislamiento (las operaciones concurrentes no producen estados inválidos) y durabilidad (un commit confirmado sobrevive a fallos). En un libro mayor, por ejemplo, el débito y el crédito de un asiento deben registrarse juntos; una transferencia no debe dejar una cuenta debitada sin el crédito correspondiente. Las restricciones, claves, bloqueos o niveles de aislamiento de la base ayudan, pero las reglas de negocio también deben codificarse y probarse. La frase "consistencia absoluta" requiere precisión: ningún estilo arquitectónico puede prometerla por nombre. La garantía debe formularse como invariantes verificables, alcance de la transacción y comportamiento ante fallos. Si la operación crítica cabe dentro de un límite de servicio y una base transaccional, es mucho más sencillo exigir ACID fuerte. Si cruza límites de servicios autónomos, coordinar un commit atómico entre bases es costoso y reduce disponibilidad o autonomía. Por ello, en esta comparación, la consistencia prioritaria significa consistencia inmediata de las invariantes financieras críticas.

5.3. Comparación de los estilos

Criterio	SBA	Microservicios
Datos y transacciones	Pocos servicios de dominio relativamente amplios; es común una base compartida o parcialmente compartida. Operaciones que afectan varias entidades pueden ejecutarse en una transacción ACID de una base, si permanecen dentro del mismo límite transaccional.	Cada servicio posee sus datos y los expone mediante API/eventos. La transacción local es ACID, pero una operación que abarca servicios requiere coordinación distribuida, saga o compensación; la consistencia suele ser eventual.
Despliegue	Los servicios de dominio pueden desplegarse por separado, aunque una base compartida, contratos comunes o cambios de esquema pueden coordinar versiones y ventanas de despliegue. Menos unidades de despliegue que MSA, por lo general.	Mayor potencial de despliegue autónomo por servicio y equipos. Requiere automatización, compatibilidad de contratos, observabilidad, seguridad, manejo de fallos y disciplina DevOps; sin esas capacidades, la independencia existe solo en teoría Fowler (2015).
Equipo mixto	Menor carga cognitiva y operacional: menos servicios, redes y almacenes que operar. Encaja mejor con experiencia heterogénea, aunque exige cuidado para evitar que la base compartida convierta servicios en módulos acoplados.	Requiere experiencia en sistemas distribuidos y operación continua. La variedad de servicios y tecnologías amplía las decisiones y los modos de fallo; una plataforma común puede reducir la carga, pero no elimina la complejidad.

5.4. Análisis por requisito

5.4.1. Consistencia de datos

La SBA ofrece una ruta más directa para transacciones financieras que involucran varias entidades: ubicar la operación crítica en un servicio de dominio y ejecutar una transacción sobre una base relacional compartida. Por ejemplo, el servicio de transferencias puede validar fondos, insertar ambos movimientos y actualizar saldos en una sola transacción. Las restricciones de unicidad e integridad referencial y claves de idempotencia complementan esta protección. Un buen diseño mantiene al servicio responsable como dueño lógico de la operación aunque varios módulos usen el mismo motor de base de datos. La MSA puede mantener ACID dentro de

cada microservicio, pero no ofrece automáticamente una transacción ACID única entre varios almacenes. Una saga divide el proceso en transacciones locales y puede ejecutar compensaciones si una etapa falla; por ejemplo, reservar fondos y luego cancelar la reserva si el servicio receptor no confirma. Esto facilita continuidad y autonomía, pero durante el flujo puede existir un estado intermedio y la compensación no siempre equivale a deshacer efectos externos. La guía de AWS describe precisamente los retos de integridad, sincronización y consistencia asociados a persistencia descentralizada Amazon Web Services (n.d.).

5.4.2. Despliegue rápido e independiente

MSA tiene ventaja potencial: modificar el servicio de reportes no obliga necesariamente a desplegar el de cuentas. Fowler identifica el despliegue independiente y la gestión descentralizada de datos como beneficios, junto con el costo de consistencia eventual Fowler (2015). Para materializar esa ventaja se requieren contratos estables, pruebas de compatibilidad, automatización de despliegue y supervisión en producción. En SBA también es posible desplegar de manera independiente los servicios de grano grueso, que en el escenario del seminario son aproximadamente entre cuatro y doce. Sin embargo, una modificación de esquema usada por varios servicios o una funcionalidad que atraviesa varios límites puede exigir coordinación. La agilidad es menor que en una MSA bien madura, pero suele ser más fácil de alcanzar con un equipo mixto.

5.4.3. Experiencia mixta del equipo

La SBA reduce el número de decisiones distribuidas y de componentes que cada integrante debe diagnosticar: menos llamadas remotas, pipelines, configuraciones y protocolos entre servicios. Eso ayuda a que desarrolladores con distintos niveles operen un sistema común. La MSA demanda capacidades sostenidas en automatización, telemetría, gestión de secretos, resiliencia, contratos y respuesta a incidentes. No es imposible con experiencia mixta, pero exige inversión en plataforma, formación y estándares de equipo.

5.5. Trade-off y recomendación

Si el CTO prioriza la consistencia de los movimientos financieros por encima de la máxima velocidad de despliegue autónomo, se recomienda comenzar con SBA: conservar juntas las operaciones que comparten invariantes ACID dentro de un servicio financiero y una base transaccional. Se acepta que algunos cambios de esquema o negocio requerirán coordinación entre servicios, y que estos no tendrán la misma autonomía que microservicios pequeños. Una evolución prudente consiste en separar primero capacidades con límites de datos claros y consistencia menos estricta (por ejemplo, notificaciones o analítica), manteniendo el libro mayor y la contabilización en el límite transaccional central. Esto no equivale a dividir cada tabla en un servicio. Si en el futuro el negocio necesita autonomía organizativa y despliegues frecuentes, se puede extraer una capacidad cuando su propietario, contrato, datos y tolerancia a consistencia eventual estén explícitos. Microservicios serían preferibles si la organización ya cuenta con madurez operativa y el requisito de independencia pesa más que la atomicidad global.

5.6. Diagramas conceptuales

5.7. Conclusión

Para estos requisitos concretos, SBA es el mejor punto de partida si “consistencia absoluta” significa que cada operación crítica debe confirmar o abortar de forma atómica. Microservicios ganan en independencia potencial de despliegue, pero ese beneficio implica aceptar coordinación, estados intermedios y mayor carga técnica, especialmente al cruzar bases de datos. La decisión correcta no es “SBA siempre” ni “microservicios nunca”: consiste en proteger el límite que

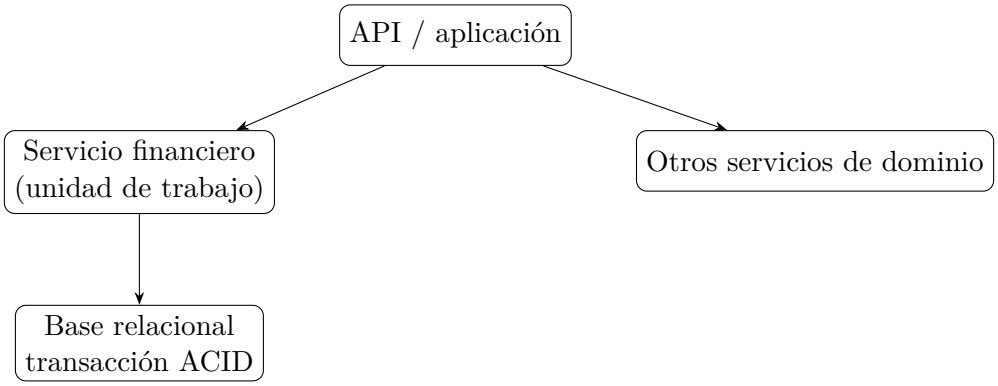


Figura 9: SBA: concentrar invariantes financieras en un límite transaccional.



Saga: transacciones locales + compensación
posibles estados intermedios

Figura 10: MSA: autonomía por servicio a cambio de coordinación distribuida.

contiene el libro mayor y separar solo las capacidades cuyo nivel de consistencia y autonomía haya sido definido de manera explícita.

6. Problema 6: El Legado de SOA y su Influencia en Arquitecturas Modernas

Enunciado. La *Service-Oriented Architecture* (SOA) con orquestación a menudo es criticada por su complejidad y por crear un cuello de botella central (el motor de orquestación). Sin embargo, algunos de sus principios, como la reutilización de servicios, siguen siendo relevantes. SOA también admite coreografía, no solo orquestación.

Analice cómo la *Microservices Architecture* aborda el problema de la reutilización de una manera fundamentalmente diferente. ¿Cómo maneja la reutilización de lógica de negocio (ej. validación de clientes) vs. la reutilización operativa (ej. logging, monitoreo)?

¿En qué escenarios un arquitecto podría todavía considerar un enfoque de orquestación (como un mediador local en microservicios o un *orchestration engine* en SOA) a pesar de sus desventajas? Proporcione un ejemplo de un flujo de trabajo complejo que se beneficiara de la orquestación explícita.

Respuesta.

La Arquitectura Orientada a Servicios (SOA) aportó una forma de organizar los sistemas alrededor de servicios que ofrecen capacidades de negocio mediante interfaces definidas. También promovió la interoperabilidad y la posibilidad de combinar servicios para construir procesos más amplios. Estos principios siguen presentes en las arquitecturas modernas. Sin embargo, los microservicios modifican el énfasis: en lugar de buscar principalmente la reutilización de servicios a escala empresarial, priorizan que cada servicio represente una capacidad de negocio delimitada y pueda evolucionar y desplegarse de manera autónoma (Richards and Ford, 2020; Richards, 2015).

6.1. Reutilización en SOA y en Microservices Architecture

En SOA, la reutilización suele plantearse desde una perspectiva empresarial. Si varias aplicaciones necesitan una misma capacidad, se procura ofrecerla mediante un servicio común. Por ejemplo, la validación de clientes podría concentrarse en un servicio de clientes utilizado por los procesos de ventas, facturación y atención. Así se evita implementar varias veces una misma regla y se facilita mantener una versión común de ella.

El inconveniente aparece cuando muchos procesos dependen de ese servicio compartido. Sus cambios, indisponibilidad o problemas de rendimiento pueden afectar a todos sus consumidores. Por tanto, aunque se reduzca la duplicación de lógica, puede aumentar el acoplamiento entre los sistemas que utilizan el servicio. La orquestación centralizada puede añadir otra dependencia si el motor que coordina los procesos se convierte en un punto obligado de comunicación o control.

Microservices Architecture adopta otro criterio: la autonomía de los servicios y la delimitación de sus responsabilidades tienen prioridad sobre la reutilización máxima. Cada microservicio debe representar una capacidad o contexto de negocio bien definido, ser responsable de sus datos y poder evolucionar sin exigir cambios coordinados en los demás servicios. Por eso, no se recomienda extraer automáticamente toda lógica aparentemente común hacia un servicio compartido. Una reutilización excesiva puede convertir ese servicio en una dependencia central y reducir la independencia que se busca con los microservicios (Richards and Ford, 2020).

Esto no significa que los microservicios prohíban reutilizar. La diferencia está en qué se reutiliza y cómo se accede a ello. Cuando una regla pertenece al contexto de clientes, por ejemplo, el servicio responsable de ese contexto debe conservar su autoridad sobre la regla y ofrecer una interfaz para que otros servicios soliciten la validación. Los consumidores no deberían acceder directamente a sus datos internos ni mantener copias independientes de la misma regla. Esta solución conserva una única responsabilidad sobre la lógica, aunque introduce una dependencia de comunicación que debe considerarse en el diseño.

6.2. Reutilización de la lógica de negocio y reutilización operativa

La reutilización de la lógica de negocio y la reutilización operativa no deben tratarse de la misma manera.

- **Lógica de negocio:** comprende las reglas y decisiones propias del dominio, como validar un cliente, calcular un precio o determinar si una operación puede aprobarse. En microservicios, estas reglas deben permanecer bajo la responsabilidad del servicio que posee el contexto de negocio correspondiente. Si otro servicio necesita utilizarlas, puede solicitar la capacidad mediante una interfaz bien definida. No conviene crear una dependencia de una biblioteca compartida de reglas de negocio que obligue a varios servicios a actualizarse y desplegarse simultáneamente cada vez que cambie una regla.

- **Lógica operativa:** comprende funciones técnicas necesarias para ejecutar y mantener los servicios, como el registro de actividad (logging), el monitoreo, la recopilación de métricas y el seguimiento de solicitudes (tracing). Estas capacidades sí se benefician de mecanismos comunes: bibliotecas técnicas, componentes de infraestructura, herramientas de observabilidad y configuraciones estandarizadas. De esta forma, los servicios mantienen prácticas operativas consistentes sin depender de otro microservicio para realizar tareas técnicas básicas.

Esta separación es coherente con los principios de modularidad y separación de responsabilidades estudiados en la asignatura. También guarda relación con los enfoques de arquitectura hexagonal, Onion y Clean Architecture, que buscan proteger la lógica central del negocio frente a dependencias de infraestructura y detalles externos (Cockburn, 2005; Palermo, 2008; Martin,

2017). La organización por capas orientada al dominio persigue, de igual manera, una separación explícita de responsabilidades (De la Torre Llorente et al., 2010).

La Figura 11 resume las diferencias entre la reutilización empresarial de SOA y el enfoque de autonomía y delimitación de responsabilidades de los microservicios:

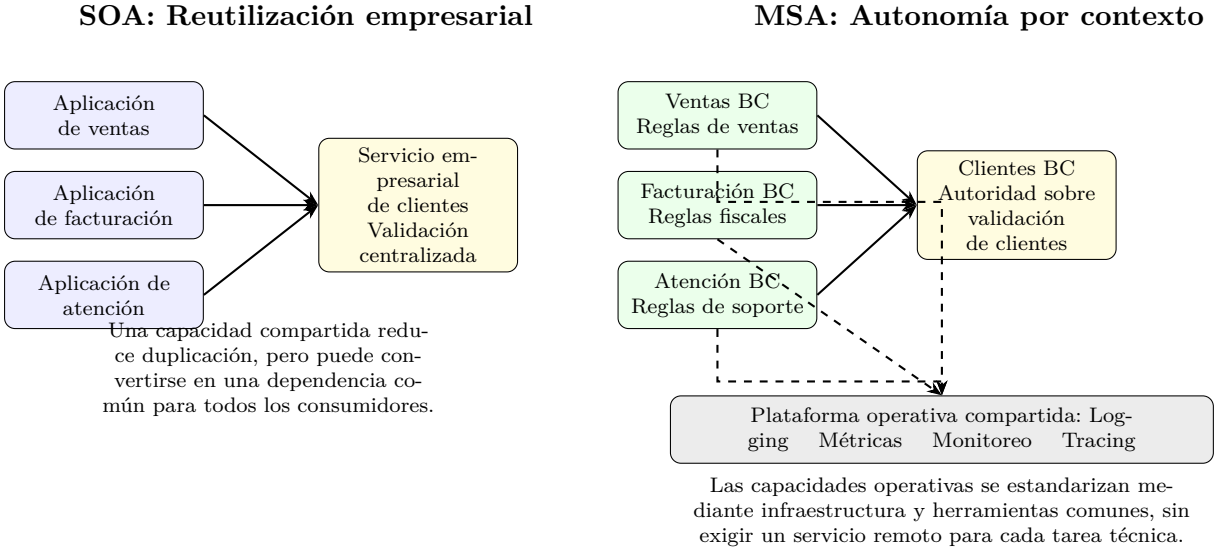


Figura 11: Diferencias entre la reutilización empresarial en SOA y la autonomía por contextos en MSA.

6.3. Cuándo sigue siendo conveniente la orquestación

Aunque la orquestación puede concentrar el control del proceso y crear un punto adicional de dependencia, sigue siendo útil cuando un flujo de trabajo tiene muchos pasos relacionados y necesita una coordinación explícita. Puede resultar conveniente cuando existen condiciones y bifurcaciones, operaciones de larga duración, reintentos, tiempos de espera, intervención humana, requisitos de auditoría o acciones compensatorias ante fallos.

En microservicios, un mediador local puede coordinar un proceso concreto sin convertirse en el coordinador de todo el sistema. En SOA, un motor de orquestación puede ser apropiado para integrar servicios empresariales heterogéneos y representar de forma centralizada un proceso que atraviesa varias aplicaciones. En ambos casos, la decisión depende de si la visibilidad y el control del flujo justifican la dependencia adicional que introduce. La orquestación no es obligatoria en SOA: también puede emplearse coreografía, donde los servicios reaccionan a los eventos de otros sin que un coordinador central dirija cada paso (Richards and Ford, 2020; Richards, 2015).

6.3.1. Ejemplo: procesamiento de un pedido

Considérese un pedido de una tienda en línea que requiere validar al cliente, reservar los productos, autorizar el pago, organizar el envío y notificar el resultado. Si el proceso se coordina mediante orquestación, un componente encargado del flujo puede indicar qué operación debe ejecutarse en cada momento y registrar el estado general del pedido.

Por ejemplo, si el inventario se reserva correctamente, el proceso continúa con la autorización del pago. Si el pago es rechazado, el orquestador puede solicitar que se libere la reserva de inventario y comunicar que el pedido no pudo completarse. Si el servicio de envío no responde, puede aplicar una política de reintentos o dejar el pedido en un estado que requiera revisión. Así, la coordinación y el tratamiento de las excepciones quedan explícitos, mientras que cada servicio conserva la responsabilidad sobre su propia lógica y sus datos.

Este tipo de flujo se beneficia de la orquestación porque no basta con emitir una notificación aislada: es necesario controlar el orden de varias operaciones, conocer el estado del proceso y definir qué hacer cuando alguna falla. El costo es mantener el componente coordinador y evitar que concentre reglas de negocio que deberían pertenecer a los servicios participantes.

La Figura 12 representa el flujo coordinado de procesamiento del pedido, incluyendo las acciones compensatorias ante un fallo:

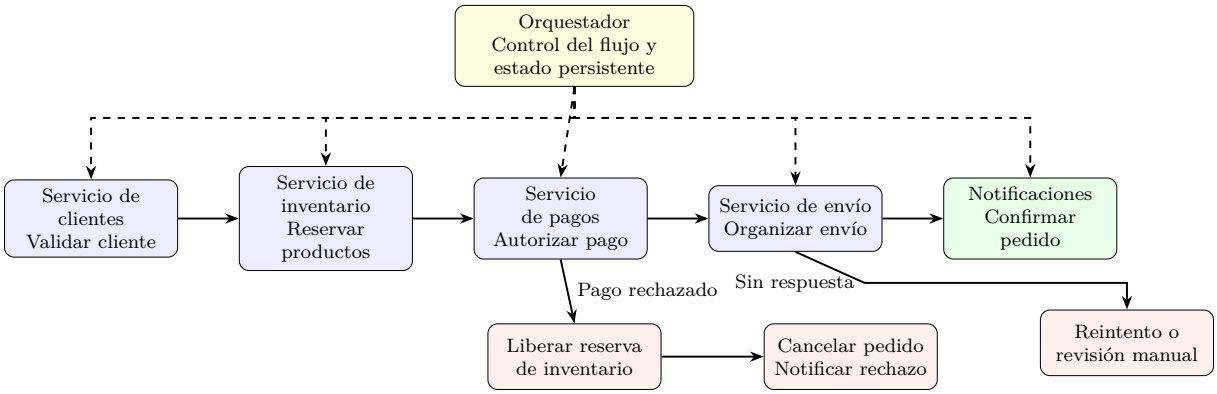


Figura 12: Orquestación del procesamiento de un pedido y gestión de excepciones.

6.4. Conclusión

El principal legado de SOA no es imponer la orquestación, sino consolidar la organización del software mediante servicios con responsabilidades e interfaces definidas. Los microservicios conservan esa idea, pero sitúan la autonomía, los límites del dominio y el despliegue independiente por encima de la reutilización empresarial indiscriminada. Por ello, la lógica de negocio debe reutilizarse respetando la responsabilidad del contexto que la contiene, mientras que las capacidades operativas pueden estandarizarse mediante mecanismos comunes de infraestructura. La orquestación continúa siendo una opción válida para procesos complejos que requieren control y seguimiento explícitos, siempre que sus beneficios compensen el acoplamiento adicional. Como ocurre con otras decisiones arquitectónicas, no existe una solución universal: deben evaluarse los compromisos entre reutilización, autonomía, complejidad y mantenibilidad según las necesidades del sistema (Richards, 2015; Richards and Ford, 2020).

Referencias

Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.

Ford, N., Richards, M., Sadalage, P., & Dehghani, Z. (2021). *Software Architecture: The Hard Parts*. O'Reilly Media.

M. Fowler, "Microservice Trade-Offs," 2015. `https://martinfowler.com/articles/microservice-trade-offs.html`

Amazon Web Services, "Enabling data persistence in microservices," AWS Prescriptive Guidance. `https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-data-persistence/`

Microsoft, "Microservices architecture style," Azure Architecture Center. `https://learn.microsoft.com/en-us/azure/architecture/microservices/`

Cockburn, A. (2005). *Hexagonal Architecture (Ports and Adapters)*.

De la Torre Llorente, C., Zorrilla Castro, U., Ramos Barroso, M. A., y Calvarro Nelson, J. (2010). *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0*. Microsoft Ibérica.

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Palermo, J. (2008). *The Onion Architecture*.

Richards, M. (2015). *Software Architecture Patterns*. O'Reilly Media.

Declaración de Uso de la Inteligencia Artificial

Durante la elaboración del presente seminario se utilizaron herramientas de inteligencia artificial generativa como apoyo complementario para la comprensión de conceptos de arquitectura de software, la elaboración de fragmentos de código y la preparación del documento técnico. Su utilización se distribuyó de acuerdo con las siguientes finalidades:

- **DeepSeek (6.2.3) y Gemini (flash 3):** se utilizaron como herramientas de apoyo para aclarar conceptos clave relacionados con los estilos arquitectónicos estudiados, sus principios de diseño y sus implicaciones prácticas. Asimismo, se emplearon para apoyar la elaboración de bloques de código y ejemplos técnicos utilizados durante el desarrollo de las respuestas.
- **ChatGPT (modelo GPT-6):** se empleó como apoyo para la revisión y modificación del código fuente en LaTeX, la organización y reformulación de determinados fragmentos del documento y la elaboración del código de los diagramas arquitectónicos mediante TikZ. Estos diagramas se diseñaron para representar las relaciones entre componentes, las direcciones de las dependencias y las consecuencias de las decisiones arquitectónicas analizadas.

Integración de los resultados y responsabilidad académica

Los resultados generados mediante estas herramientas se utilizaron como material de apoyo para la elaboración del documento y se incorporaron de acuerdo con las necesidades del seminario. Su utilización no sustituye la consulta de las fuentes bibliográficas ni la fundamentación teórica de las decisiones arquitectónicas presentadas.

El equipo asume la responsabilidad final sobre el contenido del trabajo y debe verificar la exactitud técnica de las explicaciones, la pertinencia de los ejemplos y fragmentos de código, la correspondencia entre las citas y las fuentes consultadas, y el correcto funcionamiento del documento LaTeX. Asimismo, corresponde a sus integrantes comprobar que los diagramas y las conclusiones representen fielmente los conceptos estudiados y los requisitos establecidos en el enunciado del seminario.